

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

1. Bài tập & Case Studies	4
1.1. Hướng dẫn sử dụng	4
1.2. Chương 1: Monolith → Microservices	5
1.2.1. 1-0 [NOTE] Ôn tập: Monolith vs Microservices [L1]	5
1.2.2. 1-1 [EVAL] Khi nào KHÔNG nên Microservices? [L2]	5
1.2.3. 1-2 [CASE] Case Study: Shopify — Monolith tỷ đô [L3]	6
1.3. Chương 2: DDD & Bounded Contexts	6
1.3.1. 2-0 [NOTE] Ôn tập: Domain-Driven Design Concepts [L1]	6
1.3.2. 2-1 [DESIGN] Event Storming: Thiết kế Hệ thống Đăng ký Tín chỉ [L2]	7
1.3.3. 2-2 [EVAL] Shared Library vs API — Khi nào nên gì? [L2]	7
1.4. Chương 3: Thiết kế API	8
1.4.1. 3-0 [NOTE] Ôn tập: API Design Principles [L1]	8
1.4.2. 3-1 [CASE] Case Study: Stripe API — Tại sao được coi là “gold standard”? [L2]	8
1.4.3. 3-2 [DEBUG] Debugging: API Breaking Change Incident [L3]	8
1.5. Chương 4: Giao tiếp Đồng bộ & Resilience	9
1.5.1. 4-0 [NOTE] Ôn tập: Resilience Patterns [L1]	9
1.5.2. 4-1 [DEBUG] Debugging: Cascading Failure — “Tại sao cả hệ thống chết?” [L3]	9
1.5.3. 4-2 [EVAL] REST vs gRPC vs GraphQL — Chọn gì cho scenario nào? [L2]	10
1.6. Chương 5: Giao tiếp Bất đồng bộ	11
1.6.1. 5-0 [NOTE] Ôn tập: Kafka Concepts [L1]	11
1.6.2. 5-1 [EVAL] Kafka vs RabbitMQ — Quyết định dựa trên gì? [L2]	11
1.6.3. 5-2 [DEBUG] Debugging: “Messages biến mất” — Dead Letter & Poison Pills [L3]	12
1.7. Chương 6: Saga Pattern	12
1.7.1. 6-0 [NOTE] Ôn tập: Saga Fundamentals [L1]	12
1.7.2. 6-1 [DESIGN] Design: Uber Ride Saga — Choreography hay Orchestration? [L3]	13
1.7.3. 6-2 [EVAL] Trade-off: Choreography vs Orchestration [L2]	13
1.8. Chương 7: Quản lý Dữ liệu	14
1.8.1. 7-0 [NOTE] Ôn tập: Data Management Patterns [L1]	14
1.8.2. 7-1 [EVAL] CAP Theorem trong Thực tế — “Pick Two” có đúng không? [L3]	15
1.8.3. 7-2 [DESIGN] Design: Database Migration Strategy cho KBM [L2]	15
1.9. Chương 8: API Gateway	16
1.9.1. 8-0 [NOTE] Ôn tập: API Gateway Cross-cutting Concerns [L1]	16
1.9.2. 8-1 [CASE] Case Study: Netflix Zuul → Spring Cloud Gateway → Custom Gateway [L2]	16
1.9.3. 8-2 [ADR] ADR: Rate Limiting Strategy [L2]	17
1.10. Chương 9: Bảo mật	17
1.10.1. 9-0 [NOTE] Ôn tập: Authentication vs Authorization [L1]	17
1.10.2. 9-1 [DEBUG] Security Incident: JWT Token Theft [L3]	18
1.10.3. 9-2 [EVAL] OAuth2 + OIDC Flows — Chọn flow nào cho scenario nào? [L2]	18
1.11. Chương 10: Chuyển đổi Thực tế	19
1.11.1. 10-0 [NOTE] Ôn tập: Migration Strategies [L1]	19
1.11.2. 10-1 [CASE] Case Study: Amazon — Monolith → SOA → Microservices (2002–2020) [L3]	19
1.11.3. 10-2 [EVAL] Outbox Pattern vs Event Sourcing vs Change Data Capture [L2]	19
1.11.4. 10-3 [DESIGN] Design: Data Synchronization & Strangler Fig [L3]	20
1.12. Chương 11: Observability	20
1.12.1. 11-0 [NOTE] Ôn tập: Ba Thành phần Observability [L1]	20
1.12.2. 11-1 [DEBUG] Debugging: Latency Spike Mystery [L3]	21
1.12.3. 11-2 [EVAL] Observability Stack: Build vs Buy vs Managed [L2]	21
1.13. Chương 12: Triển khai & DevOps	22
1.13.1. 12-0 [NOTE] Ôn tập: Deployment Strategies [L1]	22
1.13.2. 12-1 [EVAL] Docker Compose vs Kubernetes — At What Scale? [L2]	22
1.13.3. 12-2 [DESIGN] Design: CI/CD Pipeline cho Microservices [L3]	23

1.14. Capstone: System Design Interview – Thiết kế “Online Judge” [L3]	24
---	-----------

1.

CHƯƠNG 1.

Bài tập & Case Studies

“The best engineers don’t just know patterns, they know when NOT to use them.”
— Adapted from system design interview community

1.1. Hướng dẫn sử dụng

Mỗi bài tập/case study được phân loại:

Ký hiệu	Loại bài	Mô tả
[DESIGN]	System Design	Thiết kế hệ thống từ requirements — dạng interview
[EVAL]	Trade-off Analysis	Phân tích trade-offs giữa các phương án — dạng ByteByteGo
[CASE]	Case Study	Phân tích hệ thống thực tế (Netflix, Uber, Stripe...)
[DEBUG]	Debugging Scenario	Tìm root cause từ symptoms — dạng on-call incident
[ADR]	Architecture Decision Record	Viết ADR cho một quyết định thiết kế
[NOTE]	Ôn tập	Câu hỏi ôn tập kiến thức cốt lõi — kiểm tra hiểu bài

Mức độ: [L1] Cơ bản | [L2] Trung bình | [L3] Nâng cao (interview-level)

1.2. Chương 1: Monolith → Microservices

1.2.1. 1-0 [NOTE] Ôn tập: Monolith vs Microservices [L1]

Ghép mỗi đặc điểm (cột trái) với kiến trúc phù hợp nhất (cột phải).

Đặc điểm	Kiến trúc
A. Deploy toàn bộ hệ thống mỗi lần release	1. Microservices
B. Mỗi team sở hữu và deploy service độc lập	2. Monolith
C. Một database duy nhất cho toàn bộ ứng dụng	3. Modular Monolith
D. Module boundaries rõ ràng, nhưng deploy cùng nhau	
E. Cần đầu tư DevOps maturity cao (CI/CD, monitoring)	
F. Thích hợp cho team nhỏ (< 10 người), sản phẩm giai đoạn đầu	

Câu hỏi thêm:

- Martin Fowler khuyên chiến lược gì khi bắt đầu dự án mới? Giải thích ngắn gọn lý do.
- Kể tên 3 dấu hiệu cho thấy một monolith *đã sẵn sàng* để tách microservices (tham khảo Bảng 1.6).
- So sánh cơ chế mở rộng (scaling) giữa kiến trúc Monolith và Microservices. Giải thích tại sao Microservices tối ưu tài nguyên phần cứng hơn khi hệ thống phải xử lý tải trọng không đồng đều giữa các tính năng.

1.2.2. 1-1 [EVAL] Khi nào KHÔNG nên Microservices? [L2]

Tình huống: Người đọc trong vai architect consultant, được ba công ty mời tư vấn “có nên chuyển sang microservices?”

Công ty	Mô tả	Team	Traffic	Pain point
A	Startup fintech, 8 tháng tuổi, product-market fit chưa rõ	4 devs	200 users/ngày	Ban giám đốc muốn mở rộng quy mô lớn
B	E-commerce 5 năm, monolith 500K lines, deploy mất 4 giờ	25 devs, 4 teams	50K orders/ngày	Mỗi team deploy phải coordinate, release cycle 2 tuần
C	Hệ thống ERP nội bộ ngân hàng, Java 8, Oracle DB	10 devs, 2 teams	500 users nội bộ	Muốn “hiện đại hóa” vì đối tác tư vấn đề xuất Microservices

Câu hỏi phân tích:

- Áp dụng Decision Matrix (Bảng 10.1) cho từng công ty. Điểm số cụ thể cho mỗi tiêu chí?
- Tính **Microservice Premium** cho mỗi case: chi phí phải trả (distributed complexity, DevOps maturity cần thiết, learning curve) vs benefits thực tế
- Đưa ra recommendation cho mỗi công ty: **Yes** / **Not Yet** / **Never** — với reasoning cụ thể

4. Cho công ty B (ứng viên mạnh nhất): service nào nên tách TRƯỚC? Vì sao? Áp dụng nguyên tắc “tách service mang lại nhiều value nhất trước” (Richardson [2a, Ch.13])
 5. **Câu hỏi ByteByteGo:** Công ty A muốn người đọc thiết kế microservices “để sẵn cho tương lai”. Người đọc sẽ thuyết phục họ thế nào rằng đây là sai lầm? Dùng quote nào từ Martin Fowler?
-

1.2.3. 1-2 [CASE] Case Study: Shopify — Monolith tỷ đô [L3]

Context: Shopify (2023) vận hành trên **Ruby on Rails monolith** phục vụ hàng triệu merchants, xử lý \$444 billion GMV. Thay vì chuyển sang microservices, Shopify chọn **Modular Monolith** — tách code thành components có boundaries rõ ràng nhưng deploy cùng nhau.

Tham khảo: Shopify Engineering Blog “Deconstructing the Monolith” (2019), “Under Deconstruction” (2020).

Câu hỏi phân tích:

1. Tại sao Shopify — với quy mô lớn hơn hầu hết companies — lại KHÔNG chuyển sang microservices? Đây có mâu thuẫn với lời khuyên của Newman không?
 2. **Modular Monolith** khác gì **Microservices** về: (a) deployment boundary, (b) data isolation, (c) communication overhead, (d) team autonomy?
 3. Shopify dùng concept **Component Boundaries** thay vì **Service Boundaries**. So sánh hai khái niệm này — khi nào component boundary đủ, khi nào cần service boundary?
 4. Nếu KBM trong sách áp dụng Modular Monolith thay vì Microservices, architecture sẽ trông thế nào? Vẽ diagram. Trade-offs so với kiến trúc hiện tại?
 5. **Debate:** “Microservices là bắt buộc cho hệ thống lớn” — người đọc đồng ý hay phản đối? Dùng Shopify làm evidence.
-

1.3. Chương 2: DDD & Bounded Contexts

1.3.1. 2-0 [NOTE] Ôn tập: Domain-Driven Design Concepts [L1]

Chọn định nghĩa đúng cho mỗi khái niệm DDD:

1. **Bounded Context** là:
 - (a) Một cơ sở dữ liệu riêng cho mỗi service
 - (b) Ranh giới mà trong đó một mô hình domain có nghĩa nhất quán
 - (c) Một design pattern để tách frontend và backend
 2. **Ubiquitous Language** là:
 - (a) Ngôn ngữ lập trình chung cho toàn dự án
 - (b) Từ vựng thống nhất giữa developer và domain expert trong cùng một Bounded Context
 - (c) Tài liệu API specification bằng OpenAPI
 3. **Aggregate** là:
 - (a) Một cụm entity được xử lý như một đơn vị nhất quán (consistency boundary)
 - (b) Một bảng trong cơ sở dữ liệu quan hệ
 - (c) Một microservice chứa nhiều API endpoints
 4. Trong KBM, “bài nộp” (Submission) thuộc Bounded Context nào? “Tài khoản sinh viên” thuộc Context nào? Hai Context này giao tiếp qua cơ chế gì?
 5. Phân tích vai trò của chuyên gia nghiệp vụ (Domain Expert) trong việc xác định Bounded Context. Tại sao quá trình thiết kế hệ thống Microservices không nên chỉ do đội ngũ kỹ sư phần mềm thực hiện độc lập?
-

1.3.2. 2-1 [DESIGN] Event Storming: Thiết kế Hệ thống Đăng ký Tín chỉ [L2]

Tình huống: Người đọc đang thiết kế hệ thống đăng ký tín chỉ (LMS Course Registration) cho học kỳ mới. Stakeholders mô tả:

Scenario

“Sinh viên đăng nhập hệ thống, tìm kiếm học phần mở trong kỳ. Hệ thống kiểm tra điều kiện tiên quyết. Sinh viên chọn lớp và nhấn đăng ký. Hệ thống kiểm tra sĩ số lớp, nếu còn chỗ sẽ ghi nhận đăng ký thành công và cập nhật sĩ số. Nếu lớp đầy, sinh viên được đưa vào danh sách chờ (waitlist). Khi đăng ký xong, hệ thống gửi thông tin sang phòng Tài vụ để tính học phí. Sinh viên phải hoàn thành đóng học phí trong 7 ngày, nếu không hệ thống sẽ tự động hủy kết quả đăng ký.”

Yêu cầu:

- Liệt kê Domain Events** (≥ 12 events) theo lịch trình:
`CourseSearched` → `PrerequisiteChecked` → `RegistrationRequested` → ...
- Xác định Aggregates:** nhóm events → xác định ≥ 4 aggregates (Course? Registration? Waitlist? Payment? ...)
- Vẽ Bounded Contexts:** xác định ≥ 4 contexts với relationships
- Câu hỏi trade-off:** Nên tách “Billing/Payment” thành bounded context riêng hay gộp vào “Registration Management”? Phân tích ưu nhược của cả hai cách
- Conway’s Law:** Nếu team chỉ có 8 người, người đọc tổ chức team thế nào để align với bounded contexts?

1.3.3. 2-2 [EVAL] Shared Library vs API – Khi nào nên gì? [L2]

Tình huống: Hai microservices (Order Service và Inventory Service) cần dùng chung logic validate mã sản phẩm (product code format). Team tranh luận:

Option	Developer A đề xuất	Developer B đề xuất
Shared Library	Tạo <code>common-validation</code> JAR, cả hai services import	Để reuse, DRY principle
API Call	Tạo <code>Validation Service</code> riêng, gọi qua REST	Loose coupling, deploy độc lập
Duplicate Code	Copy validation logic vào mỗi service	Zero coupling, nhưng maintenance?

Câu hỏi phân tích:

- Phân tích trade-off chi tiết cho từng option theo 5 tiêu chí: coupling, deploy independence, performance, maintenance, team autonomy
- KBM sử dụng shared library (`kbm-shared`) chứa DTOs, ErrorCode, utils. Đây có phải anti-pattern? Khi nào shared library là hợp lý?
- Newman trong [4a] khuyên: “Chỉ share DTOs và cross-cutting concerns (logging, auth). KHÔNG share business logic.” Áp dụng nguyên tắc này: phân loại nội dung `kbm-shared` → cái nào nên giữ, cái nào nên di chuyển về service?
- Câu hỏi ByteByteGo:** Người đọc join team mới, thấy shared library 50 classes được dùng bởi 8 services. Tất cả services phải deploy cùng lúc khi shared library thay đổi. Đây là symptom của vấn đề gì? Người đọc propose giải pháp gì?

1.4. Chương 3: Thiết kế API

1.4.1. 3-0 [NOTE] Ôn tập: API Design Principles [L1]

Phân loại mỗi thay đổi API sau đây là **Breaking** hay **Non-breaking**:

Thay đổi	Breaking / Non-breaking?
A. Thêm field mới <code>createdAt</code> vào response JSON	
B. Đổi tên field <code>questionTitle</code> → <code>title</code>	
C. Thêm optional query parameter <code>?sort=asc</code>	
D. Thay đổi HTTP status từ <code>200</code> → <code>201</code> cho POST	
E. Xóa field <code>legacyId</code> khỏi response	
F. Thêm required field <code>priority</code> vào request body	

Câu hỏi thêm:

- Giải thích nguyên tắc **Tolerant Reader** (Postel's Law). Tại sao nguyên tắc này quan trọng cho API evolution?
- REST API nên dùng versioning strategy nào: URL path (`/v2/users`) hay header (`Accept: application/vnd.api+v2`)? Nêu 1 ưu và 1 nhược cho mỗi cách.
- Đánh giá tầm quan trọng của việc duy trì tính tương thích ngược (backward compatibility) khi thiết kế API trong kiến trúc Microservices so với kiến trúc Monolith. Hậu quả của việc vi phạm nguyên tắc này là gì?

1.4.2. 3-1 [CASE] Case Study: Stripe API — Tại sao được coi là “gold standard”? [L2]

Context: Stripe API được cộng đồng developer đánh giá là **API design tốt nhất thế giới**. Phân tích tại sao.

Dữ liệu: Truy cập `stripe.com/docs/api` (hoặc dựa trên kiến thức).

Câu hỏi phân tích:

- Stripe dùng **date-based versioning** (`2024-12-18`) thay vì `v1/v2`. So sánh với URL path versioning — ưu/nhược điểm gì? Khi nào date-based tốt hơn?
- Stripe API trả response dạng **envelope pattern**: `{ "object": "list", "data": [...], "has_more": true }`. So sánh với cách KBM trả `{ "data": [...], "pagination": {...} }` — cái nào tốt hơn? Tại sao?
- Stripe sử dụng **Idempotency Key** cho mọi POST request (tránh charge customer 2 lần). Thiết kế cơ chế tương tự cho `POST /api/submissions` của LMS — sinh viên submit bài 2 lần do mạng lag → chỉ chấm 1 lần
- Stripe luôn trả **error object** nhất quán: `{ "error": { "type": "card_error", "code": "expired_card", "message": "..." } }`. So sánh với error format hiện tại của LMS — gap ở đâu?
- Thiết kế:** Nếu người đọc phải thiết kế API cho tính năng mới: “Giảng viên tạo contest → sinh viên đăng ký → submit bài → leaderboard”, viết danh sách endpoints theo Stripe style

1.4.3. 3-2 [DEBUG] Debugging: API Breaking Change Incident [L3]

Tình huống incident:

Scenario

9:00 AM — Team deploy Core Service v2.3: đổi response field `questionTitle` → `title` (cleaner naming).
 9:15 AM — Mobile app (v1.2, chưa cập nhật) crash khi load danh sách câu hỏi. 200 sinh viên đang luyện tập bị ảnh hưởng. 9:30 AM — Dashboard team phát hiện, rollback Core Service về v2.2. 10:00 AM — Postmortem bắt đầu.

Câu hỏi phân tích:

1. **Root cause:** Đây là loại breaking change nào? (rename, remove, type change?) Tại sao rename field tương đương “xóa cũ + thêm mới”?
2. **Prevention:** Liệt kê ≥3 mechanisms ngăn incident này tái diễn (contract testing, schema validation, versioning policy...)
3. **Thiết kế giải pháp:** Nếu PHẢI đổi tên field, làm thế nào mà KHÔNG breaking? Mô tả quy trình 3 bước
4. **Consumer-Driven Contract Testing:** Giải thích concept. Nếu mobile app có contract test, incident này có bị bắt trước production không?
5. **ADR:** Viết Architecture Decision Record cho quyết định: “Từ nay API response chỉ được THÊM field, không được XÓA hoặc ĐỔI TÊN field”

1.5. Chương 4: Giao tiếp Đồng bộ & Resilience

1.5.1. 4-0 [NOTE] Ôn tập: Resilience Patterns [L1]

Ghép mỗi Resilience Pattern với mô tả đúng:

Pattern	Mô tả
A. Circuit Breaker	1. Giới hạn số request đồng thời đến một service
B. Retry with Backoff	2. Ngắt kết nối khi service downstream liên tục lỗi, tránh lãng phí resource
C. Bulkhead	3. Trả về giá trị mặc định khi service chính không phản hồi
D. Timeout	4. Thử lại request thất bại với khoảng cách tăng dần
E. Fallback	5. Đặt giới hạn thời gian chờ để tránh thread bị block vô hạn

Câu hỏi thêm:

1. Circuit Breaker có 3 trạng thái. Kể tên và mô tả ngắn gọn điều kiện chuyển đổi giữa chúng.
2. Tại sao timeout ở tầng Gateway phải **lớn hơn** timeout ở tầng service bên trong? Nếu ngược lại sẽ xảy ra vấn đề gì?

1.5.2. 4-1 [DEBUG] Debugging: Cascading Failure — “Tại sao cả hệ thống chết?” [L3]

Tình huống incident (mô phỏng từ thực tế):

Lịch trình**Lịch trình:**

- 14:00 – Database server của Service Tài vụ (kiểm tra công nợ) gặp sự cố hardware, tất cả queries timeout (30s)
- 14:02 – Service Đăng ký tín chỉ gọi Service Tài vụ → timeout 30s → thread pool cạn kiệt
- 14:05 – Service Cổng thông tin gọi Service Đăng ký tín chỉ → cũng timeout → thread pool cạn kiệt
- 14:08 – API Gateway gọi Cổng thông tin → timeout → Gateway thread pool đầy
- 14:10 – **TOÀN BỘ HỆ THỐNG** sập. Sinh viên không xem được cả lịch học. Health checks fail.
- 14:25 – On-call engineer restart tất cả services. Hệ thống recover sau 5 phút.

Tổng downtime – 25 phút. Ảnh hưởng: 15,000 users.

Gợi ý

Nhớ lại khái niệm Circuit Breaker và Bulkhead ở Chương 4.

Câu hỏi phân tích:

1. Vẽ **lịch trình diagram** (kiểu sequence diagram) cho cascading failure. Service nào fail trước? Domino effect lan thế nào?
2. **Root cause vs Trigger**: Database hardware failure là trigger. Root cause thực sự là gì? (gợi ý: thiếu pattern nào?)
3. Nếu hệ thống có **Circuit Breaker** (Resilience4j) giữa mỗi service call, lịch trình sẽ thay đổi thế nào? Viết lại lịch trình
4. Nếu có Circuit Breaker VÀ **Bulkhead**, kết quả khác gì so với chỉ có Circuit Breaker?
5. Thiết kế **Resilience configuration** cho hệ thống 4 tầng này: timeout, retry, circuit breaker, bulkhead cho mỗi tầng. Giải thích tại sao timeout tầng ngoài phải > timeout tầng trong
6. **Câu hỏi ByteByteGo**: Netflix xử lý 2 tỷ requests/ngày. Thiếu circuit breaker ở MỘT service có thể crash entire platform. Netflix giải quyết bằng cách nào? (Hystrix → Resilience4j)

1.5.3. 4-2 [EVAL] REST vs gRPC vs GraphQL – Chọn gì cho scenario nào? [L2]

Tình huống: Người đọc đang thiết kế communication giữa microservices cho hệ thống e-commerce:

Giao tiếp	Đặc điểm
A: Mobile app ↔ BFF (Backend For Frontend)	Bandwidth hạn chế, cần flexible queries
B: Order Service ↔ Inventory Service (internal)	High throughput, low latency, strict schema
C: Admin Dashboard ↔ Product Service	Cần query nested data (product + reviews + inventory)
D: Public API cho third-party merchants	Stability, documentation, versioning

Câu hỏi phân tích:

1. Với mỗi scenario (A-D), chọn REST / gRPC / GraphQL. Giải thích trade-off
2. Có scenario nào mà 2 protocols phù hợp như nhau? Tiêu chí “tiebreaker” là gì?
3. Netflix dùng gRPC cho inter-service communication, GraphQL cho mobile BFF. Spotify dùng gRPC internal + REST public API. Pinterest chuyển từ REST sang gRPC internal, giảm latency 50%. Phân tích: tại sao các công ty lớn đều hướng gRPC cho internal nhưng giữ REST/GraphQL cho external?

4. KBM hiện dùng REST cho LMS core, nhưng Network Lab đã có SOAP/REST/gRPC/JNP và DevOps Lab dùng Go/Chi cho một phần API. Nếu phải chọn protocol cho một boundary internal mới – chọn gì? Cost/benefit analysis?

1.6. Chương 5: Giao tiếp Bất đồng bộ

1.6.1. 5-0 [NOTE] Ôn tập: Kafka Concepts [L1]

Điền vào chỗ trống bằng thuật ngữ đúng: *Topic, Partition, Consumer Group, Offset, Producer, Broker*.

1. _____ là đơn vị logic để phân loại messages (ví dụ: `submissions` , `grading-results`).
2. Mỗi topic được chia thành nhiều _____ để xử lý song song.
3. _____ là vị trí (số thứ tự) của message trong partition – consumer dùng nó để theo dõi đã đọc đến đâu.
4. Nhiều consumer instances cùng thuộc một _____ sẽ chia nhau các partitions – mỗi partition chỉ được 1 consumer trong group xử lý.
5. _____ là thành phần gửi messages vào topic.
6. _____ là server Kafka lưu trữ và phục vụ messages.

Câu hỏi thêm:

1. Giải thích sự khác biệt giữa **at-most-once**, **at-least-once**, và **exactly-once** delivery. KBM submission pipeline nên dùng loại nào? Tại sao?
2. Khi nào nên tăng số partitions của một topic? Có rủi ro gì khi tăng quá nhiều?

1.6.2. 5-1 [EVAL] Kafka vs RabbitMQ – Quyết định dựa trên gì? [L2]

Tình huống: 4 scenarios khác nhau, mỗi cái cần message broker:

Scenario	Mô tả	Yêu cầu đặc biệt
A: Thông báo học phí	Sinh viên nộp tiền → Gửi email nhắc/xác nhận	Đảm bảo gửi đúng 1 lần, order không quan trọng
B: Lịch sử giao dịch	Mọi giao dịch đóng/rút học phí = event	Ordering đặc biệt quan trọng, retention vĩnh viễn
C: Điểm danh quét thẻ	Hàng chục ngàn sinh viên quét thẻ cổng trường	Throughput cực cao, loss vài messages OK
D: Hàng đợi xử lý file	Báo cáo đồ án: upload → queue → check đạo văn	Round-robin distribution, consumer acknowledgment

Câu hỏi phân tích:

1. Cho mỗi scenario: chọn Kafka hay RabbitMQ? Giải thích bằng architectural differences (log-based vs queue-based)
2. Kleppmann trong [7, Ch.11] giải thích: Kafka = “distributed commit log” (append-only, consumer controls offset). RabbitMQ = “message queue” (broker pushes, message deleted after ack). Cách mental model này giúp quyết định thế nào?
3. LinkedIn (inventor Kafka) xử lý 7 triệu messages/giây. Họ CẦN Kafka. KBM có nhiều loại queue khác nhau: submission pipeline, SQL Judge worker dispatch (`judge` → `judge-*`), DevOps Lab jobs. Flow nào thật sự cần Kafka, flow nào chỉ cần DB queue? Trade-off giữa “học tool đúng” vs “dùng tool phù hợp”?

4. **Exactly-once delivery:** Kafka đạt exactly-once transactional semantics từ 0.11. RabbitMQ không có native exactly-once. Giải thích tại sao exactly-once khó và tại sao “at-least-once + idempotent consumer” thường đủ
5. KBM chọn Kafka cho submission pipeline. Nếu dùng RabbitMQ hoặc DB queue thay thế, architecture thay đổi thế nào? Feature nào mất?

1.6.3. 5-2 [DEBUG] Debugging: “Messages biến mất” — Dead Letter & Poison Pills [L3]

Tình huống incident:

Symptoms

Symptoms: Kiểm tra cuối kỳ — 5 sinh viên report “nộp bài nhưng không nhận kết quả”. Giáo viên kiểm tra DB: submission records tồn tại, nhưng không có score.

Investigation log:

- Core Service logs: ✓ `SubmissionCreated` events published thành công
- Kafka topic `submissions` : ✓ Messages tồn tại (verified bằng `kafka-console-consumer`)
- Judge Service logs: 5 messages throw `JsonParseException` → consumer crash & restart → message được re-consume → crash lại → **infinite loop**
- Root cause: 5 submissions chứa SQL có ký tự Unicode đặc biệt → serialization fail

Gợi ý

Xem lại khái niệm Dead Letter Topic ở Chương 5 và cấu hình `@RetryableTopic` .

Câu hỏi phân tích:

1. Đây là example của “**Poison Pill**” message. Định nghĩa poison pill. Tại sao nó nguy hiểm hơn một message fail bình thường?
2. Không có DLT (Dead Letter Topic): consumer crash → restart → re-consume same message → crash → loop forever. **45 messages khác** (5 students không bị lỗi) cũng bị stuck sau poison pill. Giải thích mechanism (Kafka consumer offset không commit khi fail)
3. Thiết kế giải pháp **3 tầng**:
 - Tầng 1: Retry (bao nhiêu lần? backoff strategy?)
 - Tầng 2: Dead Letter Topic (message format? alert mechanism?)
 - Tầng 3: Manual review dashboard (admin xem DLT messages, reprocess hoặc discard)
4. Sau khi implement DLT, flow mới sẽ như thế nào? Vẽ diagram. 5 messages lỗi đi đâu? 45 messages còn lại có bị ảnh hưởng?
5. **Prevention:** Làm sao ngăn poison pill messages từ đầu? (Schema validation, contract testing ở producer)

1.7. Chương 6: Saga Pattern

1.7.1. 6-0 [NOTE] Ôn tập: Saga Fundamentals [L1]

Sắp xếp các bước sau đây theo đúng thứ tự của một Saga “Sinh viên nộp bài” trong KBM, và xác định compensation action tương ứng:

Các bước (chưa đúng thứ tự):

- Ghi nhận điểm vào Gradebook
- Judge Service chấm bài (chạy code trong sandbox)
- Core Service tạo Submission record
- Notification Service gửi email kết quả

Yêu cầu:

1. Sắp xếp lại 4 bước theo thứ tự đúng.
2. Nếu bước “Judge Service chấm bài” thất bại (sandbox timeout), compensation action cho bước trước đó (“tạo Submission record”) là gì?
3. Choreography Saga và Orchestration Saga khác nhau ở điểm nào? Nêu 1 ưu điểm chính của mỗi loại.
4. **Đúng hay Sai:** “Saga đảm bảo ACID transaction xuyên suốt nhiều services.” Giải thích câu trả lời.

1.7.2. 6-1 [DESIGN] Design: Uber Ride Saga — Choreography hay Orchestration? [L3]

Tình huống: Một chuyến xe Uber trải qua flow:

1. Rider request ride
2. Match driver (tìm tài xế gần nhất)
3. Driver accept
4. Trip starts (GPS tracking bắt đầu)
5. Trip ends (tính khoảng cách)
6. Calculate fare (surge pricing, discounts)
7. Charge rider's payment method
8. Pay driver (sau commission)
9. Send receipt

Nếu Step 7 fail (thẻ hết tiền):

Compensation: Cancel payment → Notify rider "add payment method" → Hold trip record (chờ retry)
 NHƯNG: KHÔNG thể undo trip (đã chạy rồi!) → Đây là "non-compensatable action"

Câu hỏi phân tích:

1. Xác định **compensable**, **pivot**, và **retriable** transactions trong 9 steps. Step nào là point of no return?
2. Uber (thực tế) dùng **Orchestration** cho ride saga — tại sao? So sánh nếu dùng Choreography: bao nhiêu events cần? Ai track trạng thái tổng thể?
3. **Isolation problem:** Rider A request ride, Driver X được match → trong lúc Driver X đang đường đến → Rider B request ride, hệ thống cũng match Driver X. Đây là anomaly gì? (Lost update? Dirty read?) Countermeasure nào cho scenario này?
4. **Timeout design:** Driver không accept trong 30 giây → timeout → tìm driver khác. Nhưng network lag → Driver accept sau 31 giây (message arrive trễ). Thiết kế mechanism xử lý “late acceptance”
5. **KBM comparison:** Hãy thiết kế Saga cho **Quy trình nộp học phí**: Sinh viên nộp tiền → Hệ thống ngân hàng báo thành công → Cập nhật công nợ → Mở khóa tài khoản đăng ký tín chỉ. Nếu bước mở khóa lỗi, quy trình compensation (hoàn tiền hoặc bảo lưu) sẽ thực hiện thế nào? Vẽ diagram.

1.7.3. 6-2 [EVAL] Trade-off: Choreography vs Orchestration [L2]

Bài tập: Cho 4 saga scenarios, phân tích nên dùng Choreography hay Orchestration:

Saga	Steps	Đặc điểm
A: User registration	Create account → Send verification email → Assign default role	3 steps, simple, low failure rate
B: E-commerce order	Reserve stock → Process payment → Ship → Update loyalty points	4 steps, payment critical, needs monitoring
C: Insurance claim	Submit claim → Fraud detection → Adjuster review → Payout → Notify	5 steps, human-in-the-loop, days-long process
D: Flight booking (multi-leg)	Book leg 1 → Book leg 2 → Book hotel → Book car → Payment	5 steps, external APIs, partial failure common

Câu hỏi phân tích:

1. Cho mỗi saga: Choreography hay Orchestration? Reasoning dựa trên complexity, number of steps, failure probability, human involvement
2. Richardson nói “≤4 steps → Choreography OK, >4 steps → consider Orchestration”. Người đọc đồng ý? Có exceptions?
3. **Saga D (Flight booking)** đặc biệt khó — external APIs (airline, hotel) có thể timeout, return ambiguous results. Thiết kế compensation cho: “Book leg 1 thành công, book leg 2 timeout — không biết thành công hay fail”. Đây là “uncertain state” — xử lý thế nào?
4. Netflix dùng **Conductor** (open-source orchestration engine). Uber dùng **Cadence/Temporal**. Tại sao các công ty lớn build orchestration engines riêng thay vì dùng choreography?

1.8. Chương 7: Quản lý Dữ liệu

1.8.1. 7-0 [NOTE] Ôn tập: Data Management Patterns [L1]

Ghép mỗi pattern/khái niệm với use case phù hợp nhất:

Pattern / Khái niệm	Use case
A. Database per Service	1. Cần audit trail đầy đủ mọi thay đổi trạng thái (ví dụ: giao dịch ngân hàng)
B. CQRS	2. Read workload và write workload có yêu cầu rất khác nhau (ví dụ: leaderboard)
C. Event Sourcing	3. Đảm bảo mỗi service có thể evolve schema độc lập
D. Shared Database	4. Team nhỏ, cần đơn giản, chấp nhận coupling giữa services

Câu hỏi thêm:

1. Giải thích ngắn gọn **CAP Theorem**. Tại sao nói “chọn 2 trong 3” là cách hiểu không chính xác?
2. KBM hiện dùng Shared Database. Nếu 2 vấn đề cụ thể mà kiến trúc này gây ra khi muốn tách microservices.

1.8.2. 7-1 [EVAL] CAP Theorem trong Thực tế — “Pick Two” có đúng không? [L3]

Tình huống: Team tranh luận về database strategy:

Scenario

“CAP theorem nói chỉ chọn được 2 trong 3: Consistency, Availability, Partition Tolerance. Vậy nếu partition xảy ra, ta phải chọn giữa C và A. Nhưng hệ thống của ta cần CẢ HAI...”

Câu hỏi phân tích:

1. Martin Kleppmann phản bác “pick two” interpretation. Ông nói CAP thực ra là: “During a network partition, choose between consistency and availability. When there’s NO partition, you can have both.” Giải thích — khác biệt thực tế so với “pick two” là gì?
2. Phân loại databases theo CAP:

Database	Classification	Behavior khi partition
PostgreSQL (single master)	?	?
Cassandra	?	?
MongoDB (replica set)	?	?
Redis Cluster	?	?

3. KBM dùng PostgreSQL cho nhiều dữ liệu giao dịch. Khi primary DB down một khoảng thời gian, hệ thống có thể unavailable cho các write path. Nếu dùng PostgreSQL với streaming replication, trade-off thay đổi thế nào?
4. **School scenario:** Giỏ đăng ký môn học dự kiến vs Thanh toán học phí — yêu cầu C/A khác nhau:
 - Giỏ môn học: availability > consistency (OK nếu rớt mạng vẫn lưu tạm, có thể thêm bớt sau)
 - Thanh toán học phí: consistency > availability (KHÔNG OK nếu trừ tiền 2 lần)
 - Thiết kế: dùng database strategy nào cho từng tính năng?
5. **Câu hỏi ByteByteGo:** Amazon.com có SLA 99.99% availability nhưng shopping cart dùng eventually consistent storage (Dynamo). Jeff Bezos nói “Customers should always be able to add items to their cart, even during failures.” Người đọc đồng ý trade-off này? Giải thích bằng CAP

1.8.3. 7-2 [DESIGN] Design: Database Migration Strategy cho KBM [L2]

Tình huống: KBM hiện có shared database — Core Service và Assignment Service dùng chung PostgreSQL. Người đọc được giao nhiệm vụ tách database.

Constraints:

- Zero downtime (sinh viên đang dùng hệ thống)
- Team 2 developers
- Timeline: 3 tháng
- Không được mất data

Tables:

Core domain: questions, submissions, scores, contests
 Assignment domain: assignments, assignment_questions, student_assignments
 Shared: users (dùng bởi CẢ HAI)
 Cross-reference: assignment_questions.question_id → questions.id

Câu hỏi phân tích:

1. Áp dụng 5 chiến lược tách database (Bảng 10.4): thiết kế timeline 3 tháng — tháng 1, 2, 3 làm gì?

2. Table `users` dùng bởi cả hai services. 3 options: (a) giữ ở Core, Assignment gọi API, (b) duplicate users data, (c) tạo User Service riêng. Phân tích trade-off mỗi option
 3. `assignment_questions` reference `questions.id` — đây là cross-domain join. Sau khi tách DB, join này KHÔNG CÒN HOẠT ĐỘNG. Thiết kế giải pháp: API composition? Data duplication? CQRS?
 4. **Dual write risk**: Trong quá trình migration, một giai đoạn cả code cũ (direct DB) và code mới (via API) cùng tồn tại. Làm sao tránh inconsistency? (Feature flag? Shadow writes?)
 5. **Rollback plan**: Nếu tháng 2 phát hiện performance issues nghiêm trọng — cách nào rollback về shared database an toàn?
-

1.9. Chương 8: API Gateway

1.9.1. 8-0 [NOTE] Ôn tập: API Gateway Cross-cutting Concerns [L1]

Từ danh sách sau, đánh dấu những chức năng nào thuộc trách nhiệm của API Gateway (có thể chọn nhiều):

1. ☐ Authentication (xác thực JWT token)
2. ☐ Business logic xử lý đơn hàng
3. ☐ Rate Limiting (giới hạn số request/giây)
4. ☐ Request routing đến đúng downstream service
5. ☐ Database migration
6. ☐ SSL/TLS termination
7. ☐ Load balancing
8. ☐ Response caching
9. ☐ Tính toán điểm bài tập (grading logic)

Câu hỏi thêm:

1. Giải thích sự khác biệt giữa **Gateway Routing** và **Gateway Offloading**. Cho ví dụ mỗi loại.
 2. Tại sao KHÔNG nên đặt business logic vào API Gateway? Hậu quả nếu vi phạm là gì?
-

1.9.2. 8-1 [CASE] Case Study: Netflix Zuul → Spring Cloud Gateway → Custom Gateway [L2]

Context: Netflix đi qua 3 thế hệ API Gateway:

Zuul 1 – (2013): Blocking I/O, servlet-based → bottleneck ở 10K concurrent connections

Zuul 2 – (2016): Non-blocking, Netty-based → chưa bao giờ được Netflix dùng rộng rãi

Spring Cloud Gateway – (2019): Reactive (WebFlux), community-driven → nhiều companies adopt

Netflix Custom – (hiện tại): Build gateway riêng optimize cho use case cụ thể

Câu hỏi phân tích:

1. Tại sao Netflix KHÔNG dùng Zuul 2 mà tự build gateway mới? (Gợi ý: organizational reasons vs technical reasons)
 2. **Blocking vs Non-blocking Gateway**: Zuul 1 dùng 1 thread/request (blocking). Spring Cloud Gateway dùng event loop (non-blocking). Với 10K concurrent connections, Zuul 1 cần bao nhiêu threads? Spring Cloud Gateway cần bao nhiêu? Implications cho memory?
 3. KBM dùng Spring Cloud Gateway cho LMS core và Go hostname-based router cho DevOps Lab. Với traffic học thuật vừa phải, blocking hay non-blocking có khác biệt? Khi nào KBM CẦN non-blocking gateway/router?
-

4. **Gateway as Single Point of Failure:** Nếu Gateway crash, toàn bộ hệ thống down. Thiết kế: high availability cho Gateway? (Multiple instances? Load balancer phía trước?)
5. **BFF Pattern (Backend for Frontend):** Netflix có gateway riêng cho mobile app (ít bandwidth) và web app (cần nhiều data hơn). Nếu KBM cần support mobile app, người đọc sẽ thiết kế BFF thế nào?

1.9.3. 8-2 [ADR] ADR: Rate Limiting Strategy [L2]

Tình huống: Hệ thống LMS bị abuse:

- Một sinh viên viết script tự động submit 1000 lần/phút (brute-force đáp án SQL)
- Judge Service overloaded → ảnh hưởng contest của 200 sinh viên khác

Yêu cầu: Viết Architecture Decision Record cho Rate Limiting.

Template ADR:

1. **Title:** Rate Limiting Strategy cho LMS API Gateway
2. **Context:** Mô tả vấn đề (abuse, impact)
3. **Decision Drivers:** Security, Fair usage, Performance, Implementation cost
4. **Options Considered:**
 - Option 1: Rate limit per user (5 submissions/phút) — cấu hình như Listing 8.3
 - Option 2: Rate limit per IP
 - Option 3: Rate limit per route + time window (contest endpoints nghiêm ngặt hơn)
 - Option 4: Adaptive rate limiting (tự điều chỉnh theo total load)
5. **Decision:** Chọn option nào? Tại sao?
6. **Consequences:** Positive, Negative, Risks
7. **Follow-up:** Monitoring strategy? Alert khi user bị rate limited?

1.10. Chương 9: Bảo mật

1.10.1. 9-0 [NOTE] Ôn tập: Authentication vs Authorization [L1]

Phân loại mỗi tình huống sau là **Authentication** (AuthN) hay **Authorization** (AuthZ):

Tình huống	AuthN / AuthZ?
A. Sinh viên đăng nhập bằng username/password	
B. Hệ thống kiểm tra sinh viên có quyền xem bài tập của lớp này không	
C. Gateway xác minh JWT token chưa hết hạn	
D. Service kiểm tra role “ADMIN” trước khi cho phép xóa user	
E. OAuth2 server cấp access token sau khi user consent	
F. API kiểm tra <code>scope</code> trong token có bao gồm <code>submissions:write</code> không	

Câu hỏi thêm:

1. JWT gồm 3 phần. Kể tên và mô tả ngắn gọn nội dung mỗi phần.
2. Tại sao sách khuyến nghị dùng RS256 thay vì HS256 cho microservices? Nêu 1 lý do chính.

1.10.2. 9-1 [DEBUG] Security Incident: JWT Token Theft [L3]

Tình huống incident:

 Report

Report: Sinh viên A phát hiện điểm bài tập thay đổi — bài chưa nộp bỗng có điểm 100. Log cho thấy submissions được tạo bởi JWT token của sinh viên A, nhưng từ IP address khác.

Investigation: Sinh viên B (cùng phòng) copy JWT token từ browser DevTools của A khi A quên log out trên máy chung. Token dùng HS256, expiry 24 giờ, không có token refresh mechanism.

Câu hỏi phân tích:

1. **Tại sao JWT stateless lại nguy hiểm trong case này?** Token bị steal nhưng server không biết — không có cơ chế revoke. So sánh với session-based auth (session ID có thể revoke ngay lập tức)
2. **Mitigation layers** — thiết kế 4 tầng bảo vệ:
 - Tầng 1: Token TTL (giảm expiry → bao nhiêu phút là hợp lý?)
 - Tầng 2: Token Refresh + Rotation (Ch.9, Listing 9.2) — giải thích tại sao rotation phát hiện được theft
 - Tầng 3: Device fingerprinting (bind token với device/browser)
 - Tầng 4: Anomaly detection (submit từ 2 IPs khác nhau trong 5 phút → flag)
3. **HS256 vs RS256:** Nếu system dùng RS256, sinh viên B có token nhưng KHÔNG có private key. Có thể tạo token giả không? Khác biệt thực tế so với HS256?
4. **Token Blacklist:** Implement token blacklist (Redis set) để revoke tokens bị steal. Trade-off: JWT “stateless” advantage bị mất. Khi nào blacklist đáng trade-off?
5. **Zero Trust:** Google BeyondCorp model — “every request must be authenticated and authorized, regardless of network location”. Áp dụng vào KBM: internal services có cần validate JWT không? (Hiện tại KBM dùng “Pragmatic Trust” — trust service-to-service calls trong boundary nội bộ)

1.10.3. 9-2 [EVAL] OAuth2 + OIDC Flows — Chọn flow nào cho scenario nào? [L2]

Bài tập: 4 applications cần integrate với Identity Provider (Keycloak/Auth0):

Application	Type	User interaction
A: Student web app (React SPA)	Single-page app, browser-based	User login, long sessions
B: CLI tool cho admin	No browser, terminal-based	Admin login
C: Judge Service (server-to-server)	No user, backend processing	Machine-to-machine
D: Mobile app (React Native)	Native app, custom login screen	User login, biometric

Câu hỏi phân tích:

1. Cho mỗi app (A-D): chọn OAuth2 flow nào? (Authorization Code + PKCE, Client Credentials, Device Authorization Grant)
2. **App A (SPA):** Tại sao Authorization Code flow cần PKCE cho SPAs? Không có PKCE → attack nào có thể xảy ra?
3. **App C (server-to-server):** Client Credentials flow không có user context. Judge Service nhận token này → biết service identity nhưng không biết user nào submit. Thiết kế: làm sao truyền user context qua service-to-service calls?

4. KBM dùng token exchange: external provider/QLĐT/username-password → KBM JWT. Nếu migrate sang Keycloak hoặc Microsoft Entra ID làm IdP trung tâm: những gì thay đổi? Lợi ích? Chi phí migration?

1.11. Chương 10: Chuyển đổi Thực tế

1.11.1. 10-0 [NOTE] Ôn tập: Migration Strategies [L1]

Sắp xếp các bước sau đây của **Strangler Fig Pattern** theo thứ tự đúng:

Các bước (chưa đúng thứ tự):

- (A) Chuyển hướng traffic từ tính năng cũ sang service mới
- (B) Xác định tính năng ít rủi ro nhất để tách trước
- (C) Xây dựng service mới implement tính năng đã chọn
- (D) Gỡ bỏ code cũ khỏi monolith sau khi service mới ổn định
- (E) Chạy song song (canary/shadow) để so sánh kết quả cũ vs mới

Câu hỏi thêm:

1. Nêu 2 anti-pattern phổ biến khi migration từ monolith sang microservices (tham khảo Phụ lục D).
2. **Đúng hay Sai:** “Nên tách tất cả services cùng lúc (Big Bang) để tiết kiệm thời gian.” Giải thích.
3. KBM nên tách service nào trước? Tại sao? (Gợi ý: xem xét độ độc lập và business value)

1.11.2. 10-1 [CASE] Case Study: Amazon – Monolith → SOA → Microservices (2002–2020) [L3]

Context: Jeff Bezos’ 2002 mandate nổi tiếng:

i Scenario

“All teams will henceforth expose their data and functionality through service interfaces. There will be no other form of inter-process communication allowed. Anyone who doesn’t do this will be fired.”

Câu hỏi phân tích:

1. Bezos mandate 2002 ra đời TRƯỚC thuật ngữ “microservices” (2014). Amazon gọi kiến trúc của họ là “Service-Oriented Architecture”. Sự khác biệt giữa SOA của Amazon và microservices hiện đại là gì?
2. Amazon không làm “big bang” rewrite. Họ tách service dần dần qua nhiều năm. Strangler Fig Pattern (Ch.10) có thể áp dụng để mô tả quá trình này? Vẽ lịch trình 3 phases
3. **“Two-Pizza Team” rule** — mỗi team 6-8 người, sở hữu 1-2 services. Conway’s Law in action? Liên hệ với Ch.2 (team topology ↔ service boundaries)
4. Amazon hiện có **hàng nghìn** microservices. Vấn đề mới: service sprawl, dependency management, ownership confusion. Giải pháp? (Service mesh, service catalog, platform teams)
5. **KBM comparison:** KBM có nhiều services/module cho team nhỏ. Áp dụng Two-Pizza Rule: bao nhiêu services là hợp lý cho team size này? KBM có risk “too many services for small team” không?

1.11.3. 10-2 [EVAL] Outbox Pattern vs Event Sourcing vs Change Data Capture [L2]

Tình huống: Team cần giải quyết dual-write problem (save DB + publish event). 3 approaches:

Approach	Mechanism	Ví dụ tool
Outbox Pattern	Write event vào DB table (cùng transaction) → poll/ CDC publish	Custom code + cron/Debezium
Event Sourcing	Events LÀ source of truth, derive state từ events	Axon Framework, EventStoreDB
CDC (Change Data Capture)	Đọc DB transaction log → auto-publish changes	Debezium + Kafka Connect

Câu hỏi phân tích:

1. So sánh 3 approaches theo 6 tiêu chí: complexity, infrastructure cost, latency, reliability, learning curve, data model impact
2. **Event Sourcing** thay đổi fundamental data model (events first, state derived). Khi nào trade-off này đáng? Khi nào quá mức? Cho ví dụ domain phù hợp (banking) vs không phù hợp (CRUD admin panel)
3. **CDC chạy bên ngoài application code** — không cần thay đổi code. Ưu điểm rõ ràng. Nhưng nhược điểm gì? (DB-specific, operational complexity, schema change sensitivity)
4. LMS chọn Outbox Pattern (Polling Publisher). Hợp lý? Nếu traffic tăng 100x, approach nào phù hợp hơn?
5. **Câu hỏi ByteByteGo:** Người đọc đang interview. Interviewer hỏi: “Giải thích dual-write problem và 3 cách giải quyết, mỗi cách cho ví dụ real-world.” — Trả lời trong 3 phút

1.11.4. 10-3 [DESIGN] Design: Data Synchronization & Strangler Fig [L3]

Tình huống: Người đọc đang áp dụng Strangler Fig Pattern để bóc tách tính năng “Điểm danh sinh viên” từ hệ thống Monolith cũ sang một Microservice mới. Tuy nhiên, tính năng “Tính điểm chuyên cần” vẫn nằm bên trong Monolith và cần dữ liệu điểm danh.

Câu hỏi phân tích:

1. Nếu Service Điểm danh ghi dữ liệu vào Database mới, làm thế nào Monolith lấy được dữ liệu này để tính điểm?
2. Phân tích trade-off của 3 phương án đồng bộ:
 - (A) Trigger ở Database mới đồng bộ ngược về DB Monolith.
 - (B) Tạo API trên Service mới để Monolith gọi đồng bộ (Sync API).
 - (C) Sử dụng Event Queue (Kafka/RabbitMQ) để Service mới gửi event `AttendanceRecorded` và Monolith hoạt động như Consumer lắng nghe.
3. Kịch bản: Quá trình chuyển đổi (Migration) kéo dài 6 tháng. Giữa chừng, trường thay đổi logic quy định thời gian điểm danh trễ. Việc maintain logic này ở cả code Monolith và Microservice tạo ra nguy cơ gì? Giải quyết thế nào?

1.12. Chương 11: Observability**1.12.1. 11-0 [NOTE] Ôn tập: Ba Thành phần Observability [L1]**

Ghép mỗi câu hỏi vận hành với thành phần Observability phù hợp nhất: **Logs**, **Metrics**, hoặc **Traces**.

Câu hỏi vận hành	Logs / Metrics / Traces?
A. “CPU usage trung bình của Judge Service trong 1 giờ qua là bao nhiêu?”	
B. “Tại sao request submit bài của sinh viên X mất 5 giây?”	
C. “Có error gì xảy ra lúc 3:00 AM khiến service crash?”	
D. “Tỷ lệ request thành công (success rate) là bao nhiêu %?”	
E. “Request đi qua những services nào và mỗi service mất bao lâu?”	
F. “Nội dung exception stack trace khi grading thất bại là gì?”	

Câu hỏi thêm:

1. Phân biệt **SLI**, **SLO**, và **SLA**. Cho ví dụ mỗi loại trong bối cảnh KBM.
2. Tại sao alert nên dựa trên SLO (trải nghiệm người dùng) thay vì metric kỹ thuật (CPU, RAM)?

1.12.2. 11-1 [DEBUG] Debugging: Latency Spike Mystery [L3]

Tình huống incident:

Alert

Alert: Grafana dashboard báo submission latency P99 tăng từ 2s → 12s. Xảy ra mỗi thứ Ba lúc 14:00.

Dữ liệu available:

- Metrics: CPU tất cả services < 30%. Memory OK. Kafka consumer lag tăng lúc 14:00.
- Logs: Không có errors. Judge Service log: “Executing SQL...” — bình thường.
- Tracing: Một trace sample cho thấy: Gateway (5ms) → Core (20ms) → Kafka publish (10ms) → Judge receive (delay 8000ms!) → Judge execute (800ms) → Core result (15ms)

Manh mối: Thứ Ba 14:00 = đầu giờ “Cơ sở dữ liệu” — 120 sinh viên access hệ thống cùng lúc.

Câu hỏi phân tích:

1. Từ trace data, **bottleneck ở đâu?** Kafka consumer delay 8000ms — nguyên nhân có thể là gì? (Gợi ý: consumer lag, partition count, consumer threads)
2. **RED analysis:** tính Rate, Error rate, Duration tại thời điểm incident. Metric nào bất thường?
3. Judge Service xử lý từng message tuần tự (single consumer thread). 120 submissions cùng lúc → queue up. Thiết kế **scaling strategy**: (a) increase consumer threads/instances, (b) increase partitions, (c) cả hai? Trade-offs?
4. Thiết kế **SLO** cho submission latency. Hiện P99 = 2s normal, spike 12s. SLO hợp lý: “99% submissions < Xs” — X bao nhiêu? Alert threshold?
5. Nếu người đọc là on-call engineer nhận alert lúc 14:05: mô tả **runbook** — 5 bước đầu tiên cần kiểm tra (từ macro → micro)?

1.12.3. 11-2 [EVAL] Observability Stack: Build vs Buy vs Managed [L2]

Tình huống: CTO yêu cầu người đọc đề xuất observability stack cho hệ thống 15 microservices:

Option	Components	Cost (ước tính/tháng)	Effort setup
A: Self-hosted OSS	Prometheus + Grafana + Loki + Jaeger	Chi phí infra tự quản	2-3 tuần
B: Managed OSS	Grafana Cloud (free tier → paid)	Theo usage	2-3 ngày
C: Commercial	Datadog / New Relic / Dynatrace	Cao hơn, đổi lấy support	1 ngày (agent install)
D: Cloud-native	AWS CloudWatch + X-Ray / GCP Cloud Monitoring	Theo cloud provider	1-2 ngày

Câu hỏi phân tích:

- Cho 3 team profiles, recommendation nào phù hợp nhất?
 - Startup 5 người, ngân sách \$0-100/tháng
 - Scale-up 20 người, ngân sách \$500/tháng, SLA 99.9%
 - Enterprise 100 người, regulated industry (banking)
- KBM hiện đã có một số mảnh Level 2 (Actuator/Prometheus/Grafana, zerolog/PLG ở DevOps Lab) nhưng chưa chuẩn hóa end-to-end. Migration path đề xuất đến Level 3: chọn option nào? Tại sao?
- Vendor lock-in:** Datadog proprietary query language vs Prometheus PromQL (open standard). Khi nào vendor lock-in chấp nhận được? Khi nào dangerous?
- OpenTelemetry (OTel) đang trở thành standard — “instrument once, send to any backend”. Chiến lược: adopt OTel từ đầu → dễ switch backend sau. Trade-off so với adopt vendor SDK?

1.13. Chương 12: Triển khai & DevOps**1.13.1. 12-0 [NOTE] Ôn tập: Deployment Strategies [L1]**

Ghép mỗi deployment strategy với đặc điểm chính của nó:

Strategy	Đặc điểm
A. Blue-Green Deployment	1. Triển khai dần cho một phần nhỏ users, tăng dần nếu OK
B. Canary Deployment	2. Duy trì 2 môi trường giống hệt nhau, chuyển traffic tức thì
C. Rolling Update	3. Gửi bản sao traffic đến version mới nhưng không ảnh hưởng user
D. Shadow Deployment	4. Thay thế từng instance cũ bằng instance mới, từng cái một

Câu hỏi thêm:

- Docker container khác gì virtual machine? Nêu 2 điểm khác biệt chính.
- Giải thích nguyên tắc **Infrastructure as Code**. Tại sao không nên cấu hình server thủ công?
- Với KBM (team nhỏ, Docker Compose), deployment strategy nào phù hợp nhất để bắt đầu? Tại sao?

1.13.2. 12-1 [EVAL] Docker Compose vs Kubernetes — At What Scale? [L2]

Tình huống: 3 hệ thống ở các scale khác nhau:

Hệ thống	Services	Traffic	Team	Yêu cầu
A: Startup MVP	3 services	500 req/ngày	2 devs	Ship fast, ngân sách thấp
B: KBM LMS + labs	Nhiều services/module	Traffic học thuật, burst khi contest/lab	3 devs	Reliability, easy ops
C: E-commerce platform	25 services	1M req/ngày	15 devs, 4 teams	Auto-scaling, zero downtime, multi-region

Câu hỏi phân tích:

1. Cho mỗi hệ thống: Docker Compose hay Kubernetes? Hoặc hybrid? Giải thích dựa trên tiêu chí: team capability, operational overhead, scaling needs
2. **Anti-pattern:** Hệ thống A (3 services, 2 devs) muốn dùng Kubernetes “để sẵn cho tương lai”. Tính **cost of Kubernetes** cho team 2 người: learning time, cluster maintenance, debugging complexity. Cost > Benefit?
3. **Migration path:** Hệ thống B (KBM) hiện dùng Docker Compose cho LMS core và k3s/Sysbox cho DevOps Lab. Khi nào CẦN mở rộng Kubernetes cho nhiều phần hơn? Xác định 3 trigger signals (traffic, team size, SLA requirements)
4. **Managed vs Self-managed K8s:** GKE/EKS/AKS vs self-setup kubeadm. Cost comparison cho hệ thống C? Risk comparison?
5. **Câu hỏi ByteByteGo:** “How does Kubernetes ensure zero-downtime deployment?” — Trả lời: Rolling Update mechanism, readiness probes, PDB (Pod Disruption Budget). Vẽ diagram

1.13.3. 12-2 [DESIGN] Design: CI/CD Pipeline cho Microservices [L3]

Tình huống: Người đọc cần thiết kế CI/CD pipeline cho KBM (Java LMS core nhiều repo/service + DevOps Lab Go multi-binary):

Requirements:

- Push code → auto build → test → deploy staging → manual approval → deploy production
- Chỉ build/test services bị thay đổi (không build lại toàn bộ)
- Database migrations chạy TRƯỚC service deploy
- Rollback mechanism: quay lại version trước trong < 5 phút
- Secret management: không hardcode passwords trong CI config

Câu hỏi phân tích:

1. **Pipeline design:** vẽ diagram CI/CD pipeline với tất cả stages. Bao gồm: changed service detection, parallel builds, test gates, approval gate, deployment
2. **Changed service detection** trong mono-repo: Git diff → xác định service nào thay đổi → chỉ build service đó. Thiết kế algorithm. Edge case: shared library thay đổi → build TẤT CẢ services?
3. **Database migration ordering:** Core Service depends on shared tables. Assignment Service depends on Core’s tables. Migration order quan trọng? Thiết kế dependency graph cho migrations
4. **Rollback strategy:** Canary deployment cho production. Nếu canary instance báo error rate > 5% → auto rollback. Thiết kế mechanism (health check → metrics → decision → rollback). Diagram?
5. **GitOps vs Push-based:** Traditional CI/CD pushes deployments. GitOps (ArgoCD) pulls from Git. So sánh cho LMS context — nên dùng cái nào?

1.14. Capstone: System Design Interview – Thiết kế “Online Judge” [L3]

Prompt (dạng interview 45 phút):

Scenario

“Thiết kế hệ thống Online Judge (như LeetCode/HackerRank) cho 10,000 concurrent users. Hệ thống cho phép: users submit code (Python, Java, SQL), hệ thống chạy code trong sandbox, so sánh output, trả kết quả. Có contest mode (1000 users submit cùng lúc) với real-time leaderboard.”

Gợi ý cấu trúc answer (theo framework system design interview):

1. **Requirements clarification** (5 phút): Functional requirements? Non-functional (latency, throughput, availability)? Scale?
2. **High-level design** (10 phút): Vẽ architecture diagram. Xác định services, databases, message queues
3. **Deep dive** (20 phút): Chọn 2-3 components quan trọng nhất:

Sandbox execution – :: Docker container per submission? Security? Resource limits?

Queue management – :: Kafka partition strategy cho fair scheduling (contest mode)?

Leaderboard – :: CQRS + Redis sorted set? Update frequency?

4. **Scale & Trade-offs** (10 phút):

- 1000 concurrent submissions → Judge Service scaling strategy?
- CAP trade-off cho leaderboard: real-time (eventual consistent) vs accurate (strong consistent)?
- Cost estimation: mỗi submission = 1 Docker container × 10 giây = how much compute?

Tiêu chí đánh giá:

- ☐ Architecture có ≥4 services rõ ràng
- ☐ Communication patterns hợp lý (sync vs async, khi nào dùng gì)
- ☐ Data management strategy (database-per-service, CQRS cho leaderboard)
- ☐ Resilience patterns (circuit breaker cho Judge, DLT cho failed submissions)
- ☐ Security (sandbox isolation, JWT auth)
- ☐ Scalability analysis (horizontal scaling Judge, partition strategy)
- ☐ Trade-off discussions (≥3 trade-offs được phân tích rõ)

Ghi chú: Các bài tập case study dựa trên thông tin public từ engineering blogs (Netflix, Amazon, Uber, Shopify, Stripe). Facts được simplify cho mục đích giáo dục — nên tham khảo nguồn gốc để có bức tranh đầy đủ.